

西电官网数据 RAG 智能检索系统

实践报告

软件工程概论课程实践 | 2026 年 06 月

项目项	内容
项目名称	西电官网数据 RAG 智能检索系统
项目定位	面向西电学生的官方公开资料问答原型
核心目标	可运行、可演示、来源可信、缺少依据时拒答
团队分工	陈天路：网站全栈开发；杨昊炜、王思帆：网站测试与 API 配置；纪依程、温阳：知识库获取与文档编写
资料来源	上传参考报告、本地 docs 文档、项目 README、源码、Plane Backlog、测试报告

目录

1. 项目概述与团队介绍
2. 需求分析模型
3. 设计模型
4. 实现过程
5. 测试策略与结果
6. AI 工具使用效果分析
7. 团队反思与总结

1. 项目概述与团队介绍

本项目面向西安电子科技大学公开官网信息，重点解决学生查询校园政策、推免/保研、生活安排、竞赛通知、教务信息和学工通知时遇到的“信息分散、检索成本高、回答缺少依据”等问题。系统通过白名单采集官方公开网页，清洗并构建本地知识库，用户以自然语言提问后，系统返回基于检索命中资料的答案与来源引用。

项目边界

首版不是通用聊天助手，也不绕过登录、VPN 或权限限制；系统只回答已收录的官方公开资料。知识库为空或相似度不足时，明确返回“未在已收录官方资料中找到依据”。

角色	主要职责	对应交付
陈天路	网站全栈开发、核心链路实现、PPT/报告整合	爬虫、RAG 服务、API、前端、最终材料
杨昊炜	网站测试、API 配置与运行验证	接口测试、演示环境配置
王思帆	网站测试、问题记录与回归验证	测试用例、缺陷反馈
纪依程	知识库获取、资料整理	官网数据源与文档素材
温阳	知识库获取、文档编写	需求/规划/总结文档

2. 需求分析模型

2.1 用户画像（AI 生成，人工校准）

需求分析阶段先由 AI 根据项目目标生成用户画像草案，再结合本地需求分析文档删去过宽的用户范围，保留与课程系统最相关的学生、教职工/辅导员、维护者和验收人员。



图 1 用户画像

2.2 用户故事地图（AI 辅助）

用户故事围绕“提出问题—检索证据—生成回答—核验来源—维护知识库”展开，并映射到 Plane Backlog 中的数据采集、知识库、RAG 问答、后端 API、前端界面、测试与文档标签。

用户故事地图（AI辅助梳理，Plane Backlog 落地）



图 2 用户故事地图

优先级	用户故事	验收标准
P0	学生希望用自然语言查询推免、竞赛、选课等信息。	返回简要回答，并展示标题、来源、发布时间和链接。
P0	当资料库无依据或问题超出公开信息边界时，用户希望系统不编造。	返回拒答提示，建议更新知识库或查看官方渠道。

P1	辅导员希望快速核验学生咨询是否有官方通知依据。	展示原文片段和可点击链接，便于转发核对。
P1	维护者希望知道知识库规模和分类分布。	`GET /stats` 返回 chunks 数量与分类统计。

2.3 用例图与用例描述

用例图（AI生成，基于需求文档修订）



图 3 系统用例图

用例	参与者	前置条件	主流程	异常/兜底
自然语言问答	学生	知识库已构建或允许空库验证	输入问题，选择分类和 top_k，提交到 /ask	无可靠命中时拒答
查看引用来源	学生/教职工	问答返回 citations	查看标题、链接、来源站点、分类、发布时间和片段	链接失效时以原文快照和提示辅助核验
更新知识库	维护者	配置 sources.json	执行 crawl 或 ingest-pages，再执行 index	抓取失败写入 crawl_failures.jsonl
查看系统状态	维护者	后端服务启动	访问 /health、/sources、/stats	后端异常时前端提示用户检查服务

3. 设计模型

3.1 系统架构图（AI 生成）

系统架构图（AI生成，按实际代码校准）

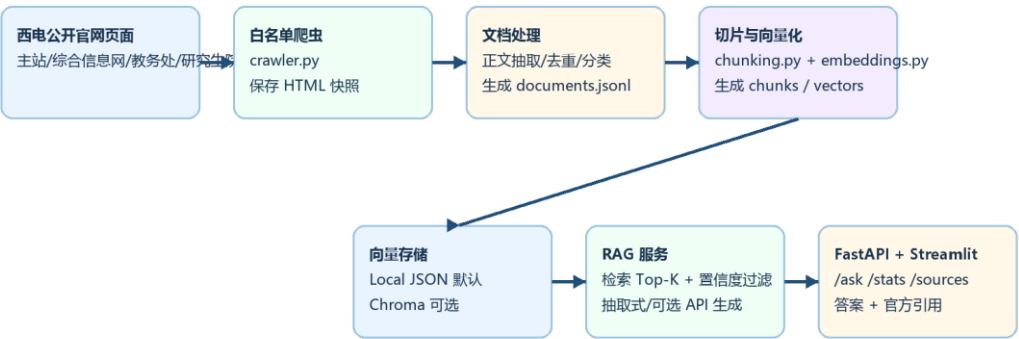


图 4 系统架构图

系统采用分层架构：数据采集层负责白名单爬取和 HTML 快照保存；知识处理层负责正文抽取、去重、分类和切片；索引检索层默认使用本地 JSON 向量索引，Chroma 作为可选后端；问答服务层负责检索、引用生成、API 生成或抽取式兜底；接口与展示层由 FastAPI 和 Streamlit 组成。

层次	主要文件	职责
数据源配置	config/sources.json	维护允许域名、种子 URL、分类关键词。
采集清洗	crawler.py	抓取公开网页，保存原始 HTML，抽取标题、正文、发布时间。
知识处理	chunking.py / embeddings.py	生成文本切片，支持本地哈希 embedding 和 OpenAI-compatible embedding。
向量检索	vector_store.py	支持 LocalVectorStore 和 ChromaVectorStore，混合向量分数与关键词分数。
问答服务	rag.py	执行 Top-K 检索、置信度过滤、引用生成、拒答和生成兜底。
接口展示	api.py / streamlit_app.py	提供 /ask、/stats 等接口与学生可用的问答页面。

3.2 数据库/存储设计（AI 辅助）

数据对象	存储位置	关键字段/内容	设计说明
数据源配置	config/sources.json	allowed_domains、seed_urls、category_keywords	集中维护官方来源，避免抓取范围失控。
原始网页快照	data/knowledge_base/pages/	doc_id.html	保留原始响应，便于复现和离线导入。
结构化文档	data/documents.jsonl	title、url、source_site、category、publish_date、content	问答前的清洗结果，当前本地样本 25 篇。
文本切片	data/chunks.jsonl	chunk_id、doc_id、text、metadata	检索的最小证据单元，当前本地样本 124 个切片。
向量索引	data/index/local_vectors.json	chunks、vectors	默认无需外部服务，适合课程演示。
失败日志	data/crawl_failures.jsonl	url、source_site、error、time	记录访问失败，不做规避。

3.3 界面原型（AI 生成）



图 5 Streamlit 问答界面原型

4. 实现过程

4.1 迭代计划与实际进展对比

迭代	计划目标	实际完成情况	偏差说明
----	------	--------	------

迭代 0	完成项目方向、角色分工、Plane Backlog 与 UML 初稿	已完成项目规划、Backlog 和需求文档	与计划一致。
迭代 1	打通官网页面到本地知识库流程	完成 sources.json、crawler、HTML 快照、documents/chunks 文件	校内通知平台若无法访问只记录失败。
迭代 2	完成检索问答与接口服务	完成 LocalVectorStore、RAG 拒答和 /ask 等接口	embedding/chat 为可选配置。
迭代 3	完成前端演示、测试和提交文档	完成 Streamlit、单元测试、AI 测试报告、PPT 和实践报告	后续可补端到端 UI 自动化。

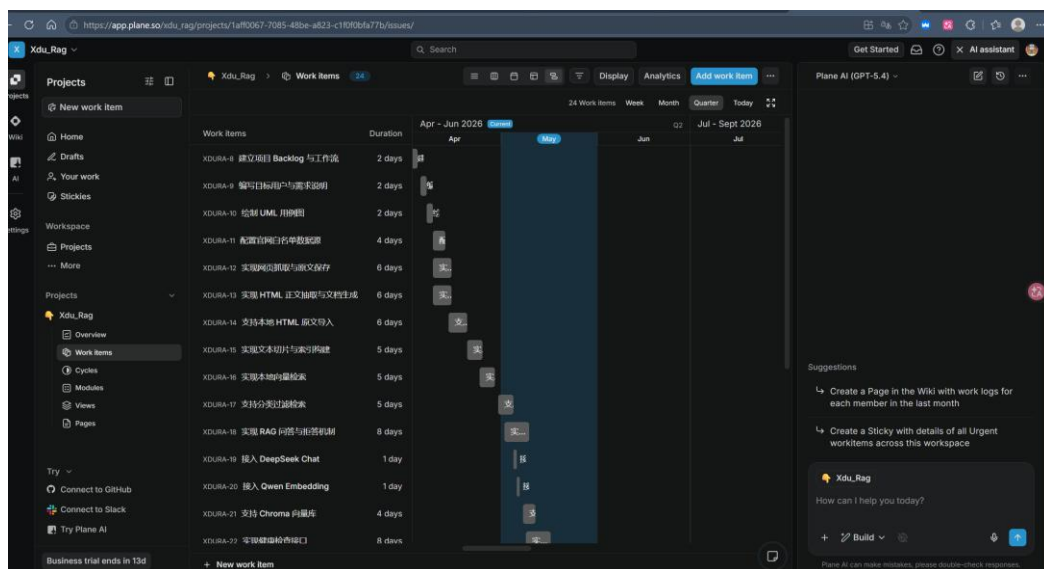


图 6 Plane 初始看板

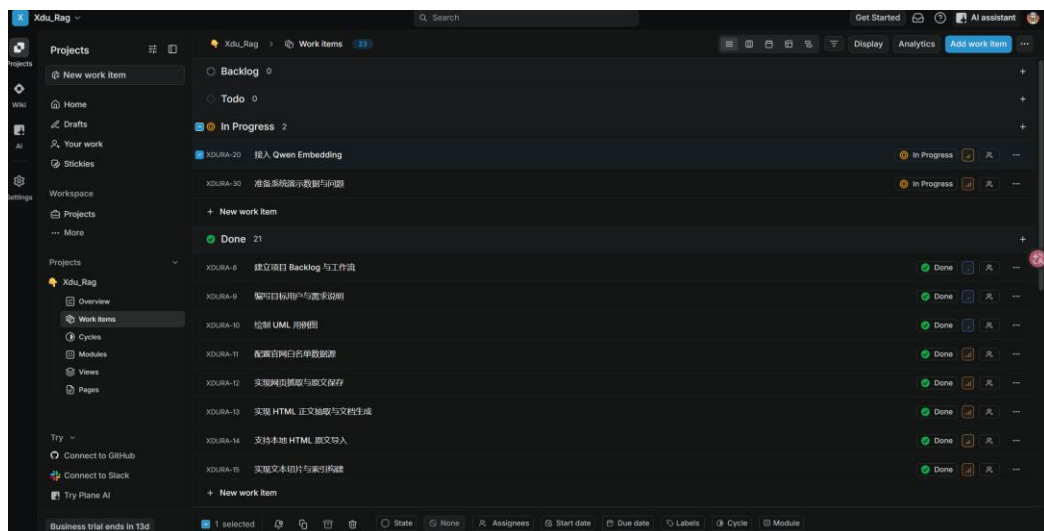


图 7 Plane 最终看板

4.2 关键代码片段（标注 AI 生成/辅助部分）

```
def split_text(text: str, max_chars: int = 700, overlap: int = 100) -> list[str]:
    clean = " ".join(text.split())
    if not clean:
        return []
    if len(clean) <= max_chars:
        return [clean]

    chunks, start = [], 0
    while start < len(clean):
        end = min(start + max_chars, len(clean))
        window = clean[start:end]
        split_at = max(window.rfind("。"), window.rfind("; "),
                        window.rfind("! "), window.rfind("? "))
        if split_at > max_chars * 0.45 and end < len(clean):
            end = start + split_at + 1
        chunks.append(clean[start:end].strip())
        start = max(end - overlap, start + 1)
    return chunks
```

说明：该片段由 AI 辅助生成后人工调整，重点适配中文标点、长文本切片和 overlap 保留上下文。最终实
现位于 src/xidian_rag/chunking.py。

```
reliable_hits = [hit for hit in hits if hit.score >= 0.08]
if not reliable_hits:
    return Answer(answer=REFUSAL, citations=[],
                  has_evidence=False, mode="refusal")

citations = hits_to_citations(reliable_hits)
# 有 API Key 时调用 Chat；失败或未配置时使用抽取式回答
```

说明：RAG 服务的核心约束是“先有证据再回答”。阈值、拒答文本、引用结构均经过人工校验，避免 AI
生成无依据政策解释。

5. 测试策略与结果

测试采用 unittest 自动化执行，并使用 AI 辅助扩展测试用例。根据本地 AI 辅助测试报告，测试从原有 7
项基础用例扩展到 94 项，覆盖 11 个核心模块，执行结果为 94/94 通过。

测试模块	覆盖重点	AI 生成比例	代表性检查
chunking	中文标点切分、短文本保留、重叠窗口	高	长文本不会截断在明显不自然的位置。
categorizer	关键词分类与得分	中	推免/推荐免试归入保研类别。
vector_store	向量构建、保存、加载、分类过滤	高	LocalVectorStore 可在临时目录完成检索。
crawler	URL 白名单、扩展名过滤、HTML 正文抽取、日期识别	高	只处理允许域名和公开页面。

rag	空库拒答、命中回答、API 失败兜底、引用去重	高	无可靠命中时返回固定拒答。
api	/health、/sources、/stats、/ask	中	请求字段校验与响应结构正确。

缺陷/问题	发现方式	修复方式	结果
中文关键词测试不稳定	AI 生成测试执行失败	将测试输入改为包含“推荐免试”等真实关键词	分类断言稳定通过
HTML 正文过短被过滤	ingest-pages 测试失败	增加测试 HTML 正文长度，满足 TEXT_MIN_LENGTH=120	导入逻辑验证通过
API 调用外部依赖不可控	测试设计评审	使用 mock 或抽取式兜底，避免真实网络依赖	单元测试可重复
Streamlit UI 未自动化覆盖	测试总结	保留人工演示验证，后续建议引入 Playwright/E2E	列为后续改进

6. AI 工具使用效果分析

6.1 各阶段 AI 工具使用清单

阶段	AI 使用方式	产出	人工校验重点
需求分析	根据题目要求生成用户画像、用户故事和用例描述草案	需求表、故事地图、用例说明	删去过宽场景，保留西电官方资料问答边界
设计建模	生成系统架构、数据流和界面原型草图	架构图、存储设计、界面原型	与实际源码目录、接口和数据文件对齐
实现开发	辅助生成切片、测试样例、接口说明和异常兜底逻辑	代码片段、README 命令、PPT 内容	运行测试，检查是否编造不存在功能
测试验证	分析源码并生成覆盖更多模块的 unittest	94 项测试用例与测试报告	审查断言是否合理，避免测试只验证实现细节
文档整理	将规划、需求、测试、PPT 内容整合成报告初稿	实践报告、图表、总结	统一口径，去掉夸张表述，补充真实限制

6.2 效率提升量化分析

任务	传统估计	AI 辅助后	效率提升	说明
需求初稿与用户故事	约 6 小时	约 2 小时	约 67%	AI 快速给出结构，人工集中做范围收敛。
Plane Backlog 拆分	约 3 小时	约 45 分钟	约 75%	AI 将 Epic 拆成可验收任务，再导入 Plane。

测试用例扩展	约 2 天	约 3 小时	约 80%+	从 7 项扩展到 94 项，覆盖 11 个模块。
PPT/报告文案整理	约 1 天	约 3 小时	约 60%+	AI 负责初稿和结构，人工校准真实性。

6.3 AI 生成内容准确率评估

内容类型	准确性评价	主要问题	处理策略
需求/用户画像	中高	容易扩展到研究生、全校多模态等超范围场景	以 SE2 和 README 为准收敛范围。
架构/流程图	高	初稿会出现不存在的数据库或服务	逐项核对 src/xidian_rag 目录和实际接口。
代码片段	中高	边界条件、路径依赖和 mock 方式需要调试	运行 unittest，失败后修正输入和断言。
测试用例	高	可能遗漏真实网络、UI、性能测试	保留单元测试优势，将网络/UI 列为后续集成测试。
文档表述	中	容易出现“全能”“毫秒级”等夸张措辞	人工删除宣传腔，改成首版原型表述。

6.4 AI 局限性与应对策略

AI 工具在本项目中显著降低了初稿、测试和文档整理成本，但它不能替代事实核验。主要局限包括：容易把系统描述得过大；无法自动判断学校内部信息是否可访问；对网络调用、UI 交互和大模型回答质量的自动化验证不足；生成的测试有时会过度贴合当前实现。应对策略是：所有结论回到源码、README 和本地文档；涉及政策回答必须保留引用；测试分层执行；文档最终由人工删减不真实的功能承诺。

7. 团队反思与总结

7.1 敏捷实践体会

通过 Plane 看板拆分 Epic、Story 和 Task，团队能把“做一个 RAG 系统”拆成可交付的小目标：先做数据源和采集，再做索引检索，再做问答接口和前端展示，最后补测试和文档。课程项目时间有限，敏捷方法最大的价值不是流程本身，而是让每个任务都有验收标准，避免最后只剩一个无法演示的概念。

7.2 AI 工具使用心得

AI 更适合作为“加速器”和“检查员”：它能快速生成结构、测试样例和文档初稿，也能帮我们发现边界场景；但项目边界、真实性、合规性和最终表述仍必须由团队负责。尤其是校园政策问答系统，可信来源和拒答机制比华丽的回答更重要。

7.3 改进建议

方向	当前不足	后续改进
检索效果	默认哈希向量适合演示，但语义能力有限	接入稳定的 embedding 服务，增加重排和时间排序。
数据更新	当前以手动 crawl/index 为主	增加增量更新、失败重试和数据源状态监控。
前端体验	Streamlit 适合演示，但交互较简单	增加历史问答、引用展开、关键词建议和错误引导。
测试体系	单元测试充分，UI/网络/性能测试不足	补充 E2E、集成测试和大规模索引性能测试。
团队协作	任务拆分已完成，但过程记录仍可更细	保留每次迭代评审记录、缺陷流转和负责人确认。

总结

本项目完成了从西电公开官网页面到本地知识库、再到带引用问答展示的首版闭环。系统最重要的实践价值在于：把 AI 能力约束在可追溯资料范围内，既能提升信息检索效率，又能避免无依据回答带来的风险。